Lecture L09

*Blind review — please score this lecture on row **L09** in the Excel form (1–10 per dimension).*

Normalization Forms and Database Schema Design (1NF–BCNF)

Learning Objectives: By the end of this lecture, you will be able to:

1. Define normalization and its importance in database design.
2. Identify and explain the different normal forms (1NF, 2NF, 3NF, and BCNF).
3. Describe the normalization process and its steps.
4. Apply the normalization process to a given database schema.
5. Explain the concept of denormalization and its trade-offs.

Introduction You're probably familiar with the "data chaos" that can occur when databases aren't designed with care. Imagine a database where duplicate data is scattered everywhere, making it difficult to query and update efficiently. This is where normalization comes in – a technique to reduce data redundancy and improve data integrity. Let's dive into the world of normal forms and learn how to tame this "data chaos"!

Normal Forms (NF)

A normal form is a set of rules that a relation must follow to ensure data consistency. Think of it like a recipe for a perfect soufflé – you need to follow the right steps to get the desired outcome. The most common normal forms are:

- 1NF (First Normal Form): A relation is in 1NF if each tuple contains only atomic values. Imagine a recipe with only basic ingredients – it's a good start, but you need more to make it great.
- 2NF (Second Normal Form): A relation is in 2NF if it is in 1NF and each non-key attribute depends on the entire primary key. This is like adding a special sauce to your dish – it enhances the flavor, but you're not quite done yet.
- 3NF (Third Normal Form): A relation is in 3NF if it is in 2NF and there are no transitive dependencies. Now you're getting closer to that perfect soufflé – the ingredients are all working together harmoniously!
- BCNF (Boyce-Codd Normal Form): A relation is in BCNF if it is in 3NF and there are no transitive dependencies. Congratulations – you've reached the perfect soufflé! But, as we'll see, this is just the beginning.

Normalization Process The normalization process involves the following steps:

1. Identify the primary key of each relation. This is like identifying the essential ingredients for your soufflé.
2. Identify the non-key attributes of each relation. These are like the add-ins that take your dish to the next level.
3. Identify the transitive dependencies in each relation. These are like the secret ingredients that make your soufflé truly special.
4. Decompose each relation into smaller relations, each in BCNF. This is like breaking down your soufflé into its individual components, making it easier to work with.
5. Combine the decomposed relations to form the final database schema. Now you can put all the pieces together to create a harmonious whole.

Example (1) Let's decompose the STUDENT relation into smaller relations, each in BCNF. We can decompose the STUDENT relation into three relations: STUDENT_ID, STUDENT_NAME, and STU-

DENT_DEPARTMENT. This is like separating the ingredients for your soufflé, making it easier to work with each component.

Example (2) Let's consider an example of a database schema for a university. We have three relations: STUDENT, INSTRUCTOR, and TEACH. The STUDENT relation has attributes such as student_id, name, and department. The INSTRUCTOR relation has attributes such as instructor_id, name, and department. The TEACH relation has attributes such as student_id, instructor_id, and course. This is like combining the individual components of your soufflé to create a harmonious whole.

Denormalization Denormalization is the process of storing the join of higher normal form relations as a base relation, which is in a lower normal form. This is like taking a shortcut through the kitchen, skipping some of the steps to get to the final product. It can be tempting to do this for performance reasons, but be careful – it can lead to data anomalies and make your life more complicated in the long run.

Conclusion In conclusion, normalization is a crucial step in designing a database, as it helps to reduce data redundancy and improve data integrity. By following the normalization process, we can ensure that our database schema is in a consistent and well-designed state. Remember, a good database design is like a perfect soufflé – it takes time, effort, and the right ingredients to get it just right!